

JavaScript Arrays - Part 1 Complete Notes

What are Arrays and Why Use Them?

Definition

- **Arrays** are collections of multiple data types in JavaScript
- You can add numbers, strings, or objects inside an array
- Different data types can exist within the same array
- Arrays allow you to store multiple values under one name

Real-world Usage Example

Arrays are commonly used in web development, especially in React components. For example:

- **Blog Website:** When displaying multiple blog cards on a website
- Each card contains similar structure (title, category, description)
- Data comes as an **array of objects** where each object represents one blog post
- Your job is to display this data on the UI using React

Creating and Accessing Arrays

Basic Array Creation

```
javascript  
  
let arr = [1, 2, 4, 5, 7];  
console.log(arr); // Shows array with 5 elements
```

Array Properties

- **Length:** `arr.length` returns the number of elements (5 in above example)
- **Type:** `typeof arr` returns "object" (arrays are objects in JavaScript)

Array Indexing

- Arrays use **zero-based indexing**
- Index positions: 0, 1, 2, 3, 4...

```
javascript
```

```
let arr = [1, 2, 4, 5, 7];  
// Index: 0 1 2 3 4  
  
console.log(arr[0]); // 1  
console.log(arr[1]); // 2  
console.log(arr[3]); // 5  
console.log(arr[4]); // 7
```

Array Mutability

Key Difference from Strings

- **Strings are immutable** - changing them creates a new string
- **Arrays are mutable** - you can directly modify the original array

```
javascript  
  
let arr = [1, 2, 4, 5, 7];  
arr[0] = 566; // This works - modifies original array  
console.log(arr); // [566, 2, 4, 5, 7]  
  
// Compare with strings:  
let a = "Harry";  
a[0] = "K"; // This doesn't work  
console.log(a); // Still "Harry"
```

Important Array Methods

1. toString()

Converts array to comma-separated string

```
javascript  
  
let arr = [1, 2, 4, 5, 7];  
console.log(arr.toString()); // "1,2,4,5,7"
```

2. join()

Joins array elements with custom separator

```
javascript
```

```
let arr = [1, 2, 4, 5, 7];  
console.log(arr.join(" and ")); // "1 and 2 and 4 and 5 and 7"
```

3. pop()

- **Removes and returns** the last element
- **Modifies** the original array

javascript

```
let a = [2, 4, 5, 6];  
let popped = a.pop(); // Returns 6  
console.log(a); // [2, 4, 5]  
console.log(popped); // 6
```

4. push()

- **Adds element(s)** to the end
- **Returns** the new length of array

javascript

```
let a = [2, 4, 5];  
let newLength = a.push(100); // Returns 4  
console.log(a); // [2, 4, 5, 100]  
  
a.push("Harry"); // Can add different data types  
console.log(a); // [2, 4, 5, 100, "Harry"]
```

5. shift()

- **Removes and returns** the first element
- **Modifies** the original array

javascript

```
let a = [1, 2, 4, 5, 7, 100, "Harry"];  
let shifted = a.shift(); // Returns 1  
console.log(a); // [2, 4, 5, 7, 100, "Harry"]
```

6. unshift()

- **Adds element(s)** to the beginning
- **Returns** the new length

javascript

```
let a = [2, 4, 5, 7, 100, "Harry"];  
a.unshift("Jack");  
console.log(a); // ["Jack", 2, 4, 5, 7, 100, "Harry"]
```

Memory Helper

- **shift** is like **pop's brother** (removes from opposite end)
- **unshift** is like **push's brother** (adds to opposite end)

Advanced Array Operations

7. delete operator

- Removes element but **keeps the memory slot**
- Length remains the same, deleted element becomes `undefined`

javascript

```
let a = ["Jack", 2, 4, 5, 7, 100, "Harry"];  
delete a[6]; // Deletes "Harry"  
console.log(a.length); // Still 7  
console.log(a[6]); // undefined
```

8. concat()

Combines multiple arrays into a new array

javascript

```
let a1 = [1, 2];  
let a2 = [3, 4];  
let a3 = [5, 6];  
let combined = a1.concat(a2, a3);  
// Returns new array: [1, 2, 3, 4, 5, 6]
```

9. sort()

- **Modifies** the original array

- Takes optional compare function for custom sorting

javascript

```
let numbers = [3, 1, 4, 2];  
numbers.sort(); // Modifies original array  
console.log(numbers); // [1, 2, 3, 4]
```

10. splice()

Most versatile method - can add, remove, or replace elements

javascript

```
let numbers = [1, 2, 3, 4, 5];  
// splice(startIndex, deleteCount, ...itemsToAdd)  
numbers.splice(1, 2); // Remove 2 elements starting from index 1  
console.log(numbers); // [1, 4, 5] (removed 2 and 3)  
  
// Adding elements while removing:  
numbers.splice(1, 3, 222, 333);  
// Remove 3 elements from index 1, add 222 and 333
```

11. slice()

Creates new array - extracts a portion without modifying original

javascript

```
let numbers = [1, 2, 3, 4, 5];  
let sliced = numbers.slice(1, 3); // From index 1 to 3 (not including 3)  
console.log(sliced); // [2, 3]  
console.log(numbers); // [1, 2, 3, 4, 5] (unchanged)
```

12. reverse()

Reverses the array **in place** (modifies original)

javascript

```
let numbers = [1, 2, 3, 4, 5];  
numbers.reverse();  
console.log(numbers); // [5, 4, 3, 2, 1]
```

Array Loops

1. Classical for loop

javascript

```
let a = [1, 3, 5, 6, 88];
for(let index = 0; index < a.length; index++) {
  ...const element = a[index];
  console.log(element);
}
```

2. forEach loop

More modern approach with callback function

javascript

```
let a = [1, 3, 5, 6, 88];
a.forEach((value, index, arr) => {
  ...console.log(value, index, arr);
});
```

forEach parameters:

- `value`: Current element value
- `index`: Current element index
- `arr`: The entire array

Key Takeaways

1. **Arrays are objects** in JavaScript (`typeof array === "object"`)
2. **Arrays are mutable** - you can modify them directly
3. **Zero-based indexing** - first element is at index 0
4. **Some methods modify original** (push, pop, splice, sort, reverse)
5. **Some methods create new arrays** (slice, concat, toString, join)
6. **Length property** shows number of elements: `array.length`
7. **Multiple data types** can coexist in same array

Advanced Loop Types

3. for...in loop

Used for iterating over **object properties** (keys)

```
javascript

let obj = {a: 1, b: 2, c: 3};

for(const key in obj) {
  ... if(obj.hasOwnProperty(key)) {
    ..... console.log(obj[key]);
  }
}

// Output: 1, 2, 3
```

Important Notes:

- `hasOwnProperty()` check ensures we only get object's **own properties**
- Filters out inherited properties from built-in objects
- VS Code auto-suggests this syntax for safety

4. for...of loop

Used for iterating over **array elements** directly

```
javascript

let a = [1, 3, 5, 7, 88];

for(const item of a) {
  ... console.log(item);
}

// Output: 1, 3, 5, 7, 88
```

Benefits:

- Simpler than classical for loop
- No need to manage index
- Can use any variable name (item, value, element, etc.)

Higher-Order Array Methods

Map Method - Creating New Arrays

Problem: Creating a new array with squared values using traditional approach

javascript

```
let arr = [1, 3, 5, 7, 11, 13];
let newArr = [];

for(let i = 0; i < arr.length; i++) {
  const element = arr[i];
  newArr.push(element * element);
}

console.log(newArr); // [1, 9, 25, 49, 121, 169]
```

Solution: Using map method (one-liner)

javascript

```
let arr = [1, 3, 5, 7, 11, 13];
let newArr = arr.map(element => element * element);
console.log(newArr); // [1, 9, 25, 49, 121, 169]
```

Arrow Function Shortcuts in Map

Multiple ways to write the same map function:

1. Full syntax with braces:

javascript

```
let newArr = arr.map((element) => {
  return element * element;
});
```

2. No parentheses (single parameter):

javascript

```
let newArr = arr.map(element => {
  return element * element;
});
```

3. No braces (implicit return):

javascript

```
let newArr = arr.map(element => element * element);
```

Map Method Parameters

Map callback receives three parameters:

javascript

```
arr.map((element, index, arr) => {  
  ... // element: current value (1, 3, 5, 7, 11, 13)  
  ... // index: current position (0, 1, 2, 3, 4, 5)  
  ... // arr: original array reference  
  ...  
  console.log('Element: ${element}, Index: ${index}');  
  ... return element * element;  
});
```

Most common usage:

- **90% of cases:** Only `element` parameter needed
- **Rare cases:** `index` and `arr` parameters used for complex logic

Map Method Features:

- **Creates new array** (doesn't modify original)
- **One-to-one mapping** - same number of elements in output
- Takes a **callback function** that processes each element
- **Arrow function shorthand:** When single parameter, parentheses optional
- **Implicit return:** No braces needed for simple expressions

Why Use Higher-Order Methods?

1. **Less code** - one line vs multiple lines
2. **More readable** - intent is clearer
3. **Functional programming** - modern JavaScript approach
4. **Immutable** - doesn't change original data

Method Comparison

Method	Purpose	Modifies Original	Returns
forEach	Execute function for each element	No	undefined
map	Transform each element	No	New array
for...of	Iterate over values	-	-
for...in	Iterate over keys/indices	-	-

Next Topics (Part 3)

Coming up:

- Filter method for conditional arrays
- Reduce method for aggregating values
- Chaining multiple array methods
- More advanced array operations

Key Takeaways from Part 2

1. **for...in** is for **objects** (keys), **for...of** is for **arrays** (values)
2. **map()** creates new arrays by transforming each element
3. **Higher-order methods** lead to cleaner, more functional code
4. **Arrow functions** can omit parentheses with single parameters
5. Always prefer **immutable operations** when possible

Note: Practice these loop types and map method to understand the difference in syntax and use cases!